

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря Сікорського»

Факультет прикладної математики

Кафедра прикладної математики

Звіт

з лабораторної роботи № 2

за темою «В⁺ дерева, індекси»

із дисципліни «Бази даних»

Виконала: Біловол Дарія

Код

```
class BPlusTreeNode:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next_leaf = None

class BPlusTree:
    def __init__(self, order=4):
        self.root = BPlusTreeNode(leaf=True)
        self.order = order

    def _find_leaf(self, key):
        current = self.root
        while not current.leaf:
            i = 0
            while i < len(current.keys) and key > current.keys[i]:
                i += 1
            current = current.children[i]
        return current

    def insert(self, key, value):
        leaf = self._find_leaf(key)
        i = 0
        while i < len(leaf.keys) and leaf.keys[i] < key:
            i += 1
        leaf.keys.insert(i, key)
        leaf.children.insert(i, value)

        if len(leaf.keys) > self.order - 1:
            self._split(leaf)

    def _split(self, node):
        mid = len(node.keys) // 2
        new_node = BPlusTreeNode(leaf=node.leaf)

        new_node.keys = node.keys[mid:]
        new_node.children = node.children[mid:]
        node.keys = node.keys[:mid]
        node.children = node.children[:mid]

        if node.leaf:
            new_node.next_leaf = node.next_leaf
            node.next_leaf = new_node

        if node == self.root:
            new_root = BPlusTreeNode()
            new_root.keys = [new_node.keys[0]]
            new_root.children = [node, new_node]
            self.root = new_root
        else:
```

```

        parent = self._find_parent(self.root, node)
        if parent is None:
            return

        index = parent.children.index(node)
        parent.keys.insert(index, new_node.keys[0])
        parent.children.insert(index + 1, new_node)

        if len(parent.keys) > self.order - 1:
            self._split(parent)

def _find_parent(self, current, child):
    if current is None or current.leaf:
        return None

    for i, c in enumerate(current.children):
        if c is child:
            return current

    for c in current.children:
        parent = self._find_parent(c, child)
        if parent:
            return parent

    return None

def search(self, key):
    leaf = self._find_leaf(key)
    for i, item in enumerate(leaf.keys):
        if item == key:
            return leaf.children[i]
    return None

def range_search(self, lower=None, upper=None):
    current = self._find_leaf(lower if lower else "")
    results = []
    while current:
        for i, key in enumerate(current.keys):
            if (lower is None or key >= lower) and (upper is None or key <=
upper):
                results.append(current.children[i])
            if upper and current.keys and current.keys[-1] > upper:
                break
        current = current.next_leaf
    return results

def delete(self, key):
    leaf = self._find_leaf(key)
    if key in leaf.keys:
        index = leaf.keys.index(key)
        leaf.keys.pop(index)
        leaf.children.pop(index)

```

```

def hash_name(name: str) -> str:
    # Розподіл букв за групами
    letter_groups = {
        1: "АБ", 2: "ВГДЕЄ", 3: "ЖЗИІЙ", 4: "КЛМН",
        5: "ОПРС", 6: "ТУФ", 7: "ХЦЧ", 8: "ЩЬ", 9: "ЮЯ"
    }

    # Створення словника для швидкого пошуку групи букви
    letter_to_digit = {}
    for digit, letters in letter_groups.items():
        for letter in letters:
            letter_to_digit[letter] = str(digit)

    name = name.upper()

    # Хешування перших трьох букв
    hash_prefix = "".join(letter_to_digit.get(ch, "0") for ch in name[:3])
    hash_prefix = hash_prefix.ljust(3, "0") # Додаємо нулі, якщо літер менше 3

    # Хешування решти слова
    remaining_hash = "".join(letter_to_digit.get(ch, "0") for ch in name[3:])
    remaining_hash = remaining_hash.ljust(7, "0")

    name_length = str(len(name)).zfill(2)
    return f"{hash_prefix}{remaining_hash}{name_length}"

if __name__ == "__main__":
    bpt = BPlusTree(order=4)
    bpt.insert("121210000005", "Агада")
    bpt.insert("332510000008", "Зайченко")
    bpt.insert("321000000003", "Іван")
    bpt.insert("552510000005", "Петро")
    bpt.insert("431310000006", "Кирило")
    bpt.insert("981000000004", "Юлія")

    print(hash_name("Дарія"))
    print(hash_name("Максиміліан"))

    print("Пошук імені 'Іван':", bpt.search("321000000003"))
    print("Пошук усіх після 'Зайченко':", bpt.range_search("332510000008"))

    bpt.delete("321000000003")
    print("Після видалення 'Іван':", bpt.search("321000000003"))

```

Приклади роботи

```
Дарія 215390000005
Максиміліан 4145343431411
Аполлінарія 1554434153911
Ігор 325500000004
Анна 144100000004
Пошук імені 'Іван': Іван
Пошук усіх після 'Зайченко': ['Зайченко', 'Кирило', 'Петро', 'Юлія']
Після видалення 'Іван': None
```

Відповіді на запитання

1. Максимальна та мінімальна кількість елементів у дереві

B+ дерево з порядком 4 означає, що кожен внутрішній вузол має від 2 до 4 дітей (мінімально $order/2$, максимальнo $order$).

- Кожен листок містить від 2 до 3 ключів ($order-1$).
- Максимальна глибина дерева – 3 (корінь + 2 рівні).

Як вирахувати максимальну кількість елементів?

Враховуючи порядок 4 і максимальну глибину 3:

1. Корінь містить 3 ключі та 4 дочірні вузли.
2. Рівень 1 має 4 вузли, кожен із яких містить 3 ключі та 4 дочірні вузли.
3. Листки (рівень 2) містять 16 вузлів, кожен із яких містить 3 ключі.

макс. елементів = (кількість листків) × (макс. ключів у листку)

$$16 \times 3 = 48$$

Отже, дерево може містити до 48 записів.

Мінімальна кількість елементів

При мінімальному заповненні вузлів:

- Корінь має 2 ключі та 3 дочірні вузли.
- Рівень 1: 3 вузли, кожен має 2 ключі та 3 дітей.
- Листки: 9 вузлів, кожен має 2 ключі.

$$9 \times 2 = 18$$

Мінімальна кількість записів у дереві – 18.

2. Параметри для 1000 чи 10000 елементів

Формула для визначення кількості записів:

макс. елементів = (кількість листків) × (макс. ключів у листку)

де кількість листків:

макс. дітей кореня × макс. дітей рівня 1

Для 1000 елементів:

Розв'яжемо рівняння:

$$M \times (M-1) \times (M-1) = 1000$$

Для $M = 10$ ($order = 10$), отримаємо:

$$10 \times 9 \times 9 = 810$$

Для **M = 11**:

$$11 \times 10 \times 10 = 1100$$

Отже, для **1000 елементів** потрібен порядок **M ≈ 11**.

Для **10 000 елементів**:

Аналогічно:

$$M \times (M-1) \times (M-1) = 10000$$

При **M = 32**:

$$32 \times 31 \times 31 = 30752$$

Отже, для **10 000 елементів** порядок дерева має бути **M ≈ 32**.

3. Особливості хеш-функції

Хеш-функція зберігає алфавітний порядок, що дає такі переваги:

- **Швидкий діапазонний пошук** – можна миттєво отримати всіх, чиє ім'я йде після або перед певного значення.
- **Рівномірний розподіл** – хеш уникає скупчення даних в одній групі, що зменшує глибину дерева.
- **Можливість фільтрації** – можна швидко знайти прізвища певної довжини або букви.

4. Чи обов'язкова спеціальна хеш-функція?

Необов'язково, але без хешування довелося б зберігати довгі строки у вузлах і порівнювати їх побітово, що значно сповільнило б пошук. Хешування прискорює пошук, дозволяючи використовувати числові порівняння замість повного порівняння рядків.